A Review of Monitoring Techniques for Service Based Applications

Sridevi Saralaya, Rio D'Souza

Dept. of Computer Science and Engineering, St. Joseph Engg. College
Vamanjoor, Mangalore 575028, India
sridevisaralaya@yahoo.com
rio@ieee.org

***Abstract*— A Service-Based Application (SBA) is composed of a number of loosely coupled services available on the network which provide the desired functionalities. Service Based Applications execute in dynamic business environments and have to address evolving requirements. Hence they should be flexible to identify violations and adapt to changes in business requirements or context. Monitoring is the key element for adaptation. A Service Based Application can be viewed in terms of three layers i.e., Business Process Management Layer, Service Composition Layer and Service Infrastructure Layer. Application performance depends on the combined performance of components and their interactions within the SBA layers. Therefore it necessitates to constantly monitor the health of the application by monitoring activities occurring in SBA layers. In this paper we present a view of the monitoring approaches across the three layers.**

Keywords-Service-Based Application; Monitoring; Business Activity Monitoring; Composite Service; Service-Infrastructure

1. INTRODUCTION

Service-Oriented Computing (SOC) paradigm refers to the set of concepts, principles, and methods that represent computing in Service-Oriented Architecture (SOA) which utilises independent component services with standard interfaces as the basic construct. Services are autonomous, platform independent computational entities that can be used in heterogeneous environments. As Service Based Applications (SBA) run in dynamic business environments they have to manage constantly evolving requirements. The Service based applications should be able to identify and react to changes in business requirements and application context. Developers must be able to continuously monitor the status and behaviour patterns of loosely coupled services in the application. Hence monitoring and adaptation can be viewed as an important function of the SBA.

An SBA can be viewed by its three functional layers as Business Process Management (BPM) Layer, Service Composition (SC) Layer and Service Infrastructure (SI) Layer [1]. BPM is the top most layer in which the whole business process is viewed as a workflow with business activities as constituent services. The intermediate SCC layer accomplishes the construction of SBA by composing appropriate services to accomplish workflow specified in the BPM layer. At the bottom level we have SI layer which constitutes the run-time environment of SBA. In this paper we aim to study the monitoring approaches in each functional layer.

Remainder of this paper is organized as follows: The next section provides a classification of the monitoring techniques; section 3 reviews monitoring in the Business Process Management layer; in section 4 we consider Service Composition layer and Service Infrastructure in Section 5. We conclude in Section 6.

2. MONITORING TECHNIQUES

A monitor observes the behaviour of a system and determines if it is consistent with a given specification [2]. According to Plattner *et al*. [3], a monitor is a program which observes execution of another program. Monitoring may be performed for various reasons such as discovery of problems in application execution, run-time optimization, detect relevant contextual changes and events, collect information relevant for application evolution such as histories of adaptation, decisions made in different executions to support evolution and governance of SBA, etc.

2.1 Classification of Monitoring Aspects

We can classify existing approaches to monitoring SBA based on several dimensions [4]-[6]. Such a classification (Fig 1) may be considered as an answer to some basic questions like:

- why should monitoring be performed (monitoring task) ;
- what information should be monitored;
- how should monitoring be performed (implementation methods and techniques);
- who are involved in the monitoring process

2.1.1 *Monitoring Task*

The monitoring task specifies the purpose or goal of monitoring activity. Some purposes identified in literature are:

- *Run time analysis* of system correctness i.e. run-time verification of behavioural or non-functional properties, run-time conformance analysis, analysis of service-level agreements and requirement analysis
- To perform *diagnosis and repair* from faulty state
- To perform *optimization* of resources
- *Adaptation* to contextual changes, or recovery from failure
- *Evolution* to support long term adaptation

978-0-7695-5127-2/13 $31.00 © 2013 IEEE
DOI 10.1109/ADCONS.2013.18

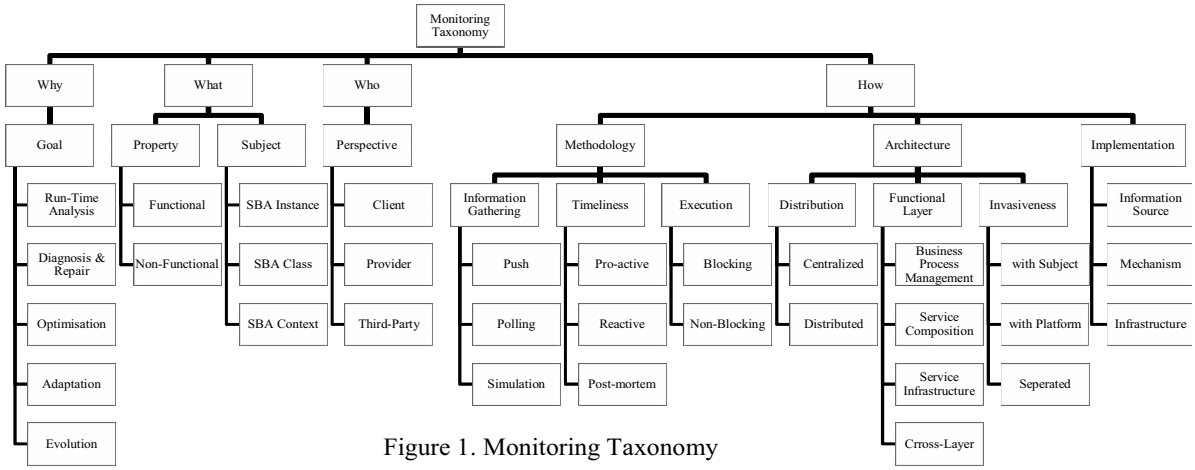

Figure 1. Monitoring Taxonomy

2.1.2 Specification of Monitoring Information

The specification of monitoring information deals with what information is to be monitored. The key issues considered here are:

- monitored *property* and
- *subject* of monitoring.

The monitored property may be functional or non-functional. Functional property refers to service operations, their signatures, binding information and behavioural specifications of the SBA. Non-functional properties also known as Quality of Service (QoS) properties may be availability, performance, reliability or accessibility. The subject or scope of monitoring may be an SBA instance, an SBA class or the SBA context.

2.1.3 Implementation Methods and Techniques

The monitoring methodology, architecture and different implementation techniques have a very broad scope. We discuss some important aspects here.

2.1.3.1 Monitoring Methodology

The monitoring methodology defines the characteristics of the monitoring process itself. The key issues here are:

- Information gathering
- Timeliness
- Execution
- Monitoring Techniques

Information gathering refers to the approach used to collect and aggregate data from various information sources which may be in push or polling mode. Timeliness is the time difference between the moment when the event occurs and the moment when it is actually reported to the monitor. Here we can distinguish between reactive, post-mortem and proactive approaches. Execution of the monitoring process with respect to the monitoring subject may be synchronous (blocking) or asynchronous (non-blocking). In synchronous mode the monitoring subject is blocked until all monitoring measurements are performed whereas in asynchronous mode the monitoring process runs in parallel with execution of the monitoring subject. Monitoring techniques refer to particular solutions employed in order to provide the above characteristics such as data or process mining approaches, database monitoring, automata-theory approaches etc.

2.1.3.2 Monitoring Architecture

Monitoring architecture refers to the way the monitoring framework is structured and decomposed. The main characteristics of the architecture are:

- Functional SBA layers addressed
- Distribution
- Invasiveness

The monitoring framework may consider elements provided in individual functional SBA layers i.e. BPM, SC or SI layer and be termed as *vertical* structuring of framework or may involve a set of components across functional layers known as cross-layer monitoring. Distribution refers to *horizontal* structuring of the monitoring framework, which specifies how monitoring components are logically and physically located i.e. centralized or distributed. In a centralized architecture the monitoring components are located in a single node whereas in a distributed architecture the components are distributed across the network. The degree of invasiveness refers to how tightly a monitoring framework is integrated with the monitoring subject, or the platform in which a subject operates.

2.1.3.3 Monitoring Implementation

Implementation defines the way in which the monitoring methodology and architecture are realized. It is characterized by:

- Information Sources
- Realization mechanism
- Monitoring Infrastructure

Information sources refer to components and entities that provide data used by the monitor in order to evaluate monitoring properties such as log files, messages, events, timers, sensors and probes. Realization mechanism defines the tools and facilities required to enable a given monitoring methodology, to implement the monitoring technique and build monitoring architecture such as aspect oriented programming or complex event processing etc. Monitoring Infrastructure refers to tools and facilities for monitoring

frameworks such as APIs for deploying and executing the monitoring code.

2.1.4 Participants/Stakeholders involved in the Monitoring process

The *who* dimension characterises the different perspectives involved in the monitoring process:

- Client
- Provider
- Third-party

Client perspective is the outside view of the system aiming to check whether it meets customer requirements. Provider perspective sees the system from the inside i.e. satisfying the expectations of the owner. Third-party view takes an independent view on the monitoring subject.

3. MONITORING IN BPM LAYER

According to Aalst *et al.*, [7] Business Process Management (BPM) includes methods, techniques, and tools to support the design, enactment, management, and analysis of operational business processes. Monitoring in BPM is called Business Activity Monitoring (BAM). Analysts at Gartner group [8] coined BAM as the concept of providing real-time access to critical business performance indicators to improve the speed and effectiveness of business operations. BAM provides information on Key Performance Indicators (KPIs). In BAM, monitored activities are instances of business processes which are realized using BPM technologies such as Business Process Execution Language (BPEL). BAM is based on event processing. Events are collected from diverse sources such as Enterprise Resource Planning (ERP) systems, databases and legacy applications. The collected events are composed and correlated using Complex Event Processing (CEP) Technologies [9]. Events can be correlated with historical data to achieve a holistic view of business activities so as to support decision making [10]. Below, we discuss some studies which address BAM in BPM layer.

3.1 Platform for Intelligent Business Operation Management

Castellanos *et al.* [11] have developed a platform for business operations management that (i) provides visibility into the business process being executed across many systems (ii) allows users to define metrics and (iii) predicts the future values for a metric on a process execution. In order to provide visibility into processes, the authors have developed an Abstract Process Monitor (APM) component which allows users to model and manage abstract processes which provide different views of an actual business process, by interpreting events occurring in IT infrastructure. Users can not only define metrics to measure characteristics of process instances, processes, resources and business operations but also specify how the metrics should be computed. A template based approach is used for metrics computation. For analysis and prediction of metrics, data mining techniques based on decision trees are employed.

3.2 Process-Based Performance Measurement Model

Han *et al*. [12] propose a BAM framework in which the KPI's are co-related to business processes. They state that performance measurement should not be independent of business processes. The PBPM is classified into three levels which are Strategic Level KPI (SLK), Tactical Level KPI (TLK) and Operational Level KPI (OLK). In the KPI model, the contribution of a specific KPI to other KPI's on the same or higher level is based on a 3-point scale. Similarly in the business process model, business processes are classified into three levels which are enterprise level, process level and sub-process level. Finally, the relation or influence of a business process on a specific KPI is given by the K-P model. SLK, TLK and OLK in the KPI model relates to enterprise, process and sub-process. The information is displayed in a dashboard which helps in analysis.

3.3 Ontology based metric computation for BPA

Pedrinaci *et al*. [13] propose a domain independent ontology which supports definition of business metrics and a computation engine which interprets and automatically computes the metrics over domain specific data. The metrics ontologies help the business analysts to specify and compute metrics based on the information gathered from the low level audit trails for analyses and managing business processes in a domain independent way. Two types of metrics are defined – functional metrics and aggregate metrics. Metric computation takes a metric as input and returns a quantitative analysis result. Functional metric computation is a two-step process: the input roles are evaluated and then the metric is computed. Whereas computation of aggregate metrics is a three-step process computed by applying an aggregate function on a selected population. Calculating population filter is the first step followed by computation of function metrics for each individual before applying the aggregation construct. In the last step the quantitative analysis is returned by applying aggregate function to the resulting population.

3.4 Defining Process Performance Indicators

Ortega *et al*. [14] use ontologies for defining PPI's and specify relationship between PPIs and the elements of business processes. The PPI's are based on two criteria: (i) single instances or aggregation of a set of instances; (ii) the method to obtain the value of a measure as base measure, aggregated or derived measure. Base measure values are obtained from a single process instance whereas aggregated measures are obtained by applying aggregate constructs on a set of measures to get a single value. Derived measure is a mathematical function of either several different process measures or several different measures from the same process instance. Dependencies between measures can be positive or negative depending on how changes in one measure affect the other. SWRL rules are used to infer dependencies, and knowledge inferred from them is used to answer queries concerning PPI's of an organisation.

3.5 Agent based approach for analysing Business Processes

Jeng *et al.* [15] describe agent based architecture for providing continuous, real-time analytics for business processes. Analytical processing is performed by an agent framework that detects exceptions in business environment and performs complex analytical tasks to reflect the gap between present situation and desired goal. The architecture is made up of Presentation layer, BI agent layer, Event

Processing Container (EPC) and Integration layer. The integration layer is responsible for extracting events from business processes and workflow systems and provide them to the EPC which handles workflow events in real-time. The incoming process events are transformed into metrics which are stored in the process data store. EPC publishes events to the BI agent layer for analysis. The BI agent layer is made up of sensing, reactive, deliberate, reflective and responsive agents. The sensing agents capture events and provide them to reactive, deliberate and proactive agents, which analyse and react to exceptions in business environment governed by management policies. Based on directives from the agent layers, the response agent generates action output into the business environment. The approach is demonstrated on a supply chain management use case for microelectronics manufacturing.

4. Monitoring in Service Composition Layer

Service composition is a realization mechanism of the business model from the BPM functional SBA layer, which integrates a set of services. Composition monitoring targets the analysis of composition correctness or evaluation of its properties. The properties may have various forms and target different monitoring objectives. Some of the monitoring scenarios include: monitoring for diagnosis, monitoring for SLA compliance, composition optimisation, monitoring correctness of service composition etc.

4.1 Monitoring for Diagnosis

Diagnostic monitoring of faults occurring in a system and performing such actions as that are required to resolve the same so as to return the system back to normal execution is a very important aspect of monitoring for diagnosis.

4.1.1 Fault Tolerant Orchestration by means of Diagnosis

A web service orchestration framework is presented by Ardissono *et al.* [16] for the purpose of diagnosis of exceptions and subsequent execution of exception handlers. The framework is based on Model Based Reasoning in which the exceptions are analysed in a component based way by providing local diagnosis for individual services and a global diagnostic engine to combine the effect of local diagnosis. The local diagnose associated to each web service generates diagnostic hypotheses for exceptions occurring in the local web service. The global diagnose generates the global diagnostic hypothesis by combining the entire local hypothesis. The identified global hypothesis represents the cause of the problem and is used for fault handling which is achieved by making the local fault handling diagnosis aware by considering the hypothesis in the recovery actions.

4.2 Monitoring for SLA Compliance

Service-Level-Agreement (SLA's) are contractual agreements between providers and consumers, which govern the exact conditions (both functional and non-functional) under which the services should be delivered. Monitoring for SLA compliance aims to check whether service functionality meets the requester's expectations. Violations of SLA may result in monetary penalties for the provider. Therefore the providers generally aim to prevent SLA violations.

4.2.1 Automated SLA Monitoring for Web Services

Sahai *et al.* [17] propose an automated and distributed SLA monitoring engine. Automation of SLA monitoring is done by a specification language that enables definition of precise and flexible SLA. The authors argue that flexibility is required since all possible SLA's cannot be understood or anticipated, which helps to create a generic SLA management system. The main elements of the SLA specification are mesauredItem, measuredAt (client or provider side), event on which the measurement should be done and instructions for evaluating the value. At the service level, data is collected by proxies that intercept messages and perform calculations. At the business process level, the information about internal process activities is stored in process logs. The service level monitoring engine coordinates the monitoring actions, creates new monitoring instances based on the duration as specified in SLA and associates the monitoring information with the monitored objects. Distributed monitoring is supported if measurement is to be performed at both the consumer and provider side. The information is exchanged via measurement exchange protocol which takes care of transferring measurements at the right level of aggregation and at the right frequency.

4.2.2 CREMONA

An agreement management architecture for creation, management and monitoring SLA's represented as WS-Agreement documents which is independent of any application domain has been proposed by Ludwig *et al.* [18]. Cremona provides a layered model for defining, creating, binding and monitoring of contracts. The monitoring module not only monitors and detects contract violations but also predicts future violations thus helping in corrective actions in advance.

4.2.3 Monitoring WS-Agreements

Mahbub *et al.* [19] have developed a framework to support monitoring of functional and QoS requirements specified as part of SLA's. EC-Assertion is an event calculus based language used for the specification of service guarantee terms which support descriptions of operational context of an agreement, the monitoring policy and the functional and quality requirements for services. The framework is made up of a monitoring manager, an event receiver, an event database, a deviation database and a monitoring console. Two types of events are used in order to detect deviations; they being recorded events which are events captured at run-time and derived events which are obtained from recorded events by deduction. The monitoring process relies on automated extraction of templates which represent formulas in event calculus. The deviation of events, the reasoning schema and run-time analysis are based on the inference rules specified in first order logic.

4.3 Monitoring for composition optimisation

Monitoring for composition optimisation refers to continuous monitoring of involved quality properties of services involved in the composition, in order to detect changes or violations which may necessitate adaptation and make optimal decisions over the composition configuration. The monitoring process continuously observes and evaluates

the value of a certain utility function, which aggregates weighted measures of single quality properties of the involved services to calculate composition optimality.

4.3.1 QoS Aware Replanning for Composite Services

Canfora *et al*. [20] have proposed a proxy approach to be employed for replanning and dynamic binding. During composite service execution, when the system predicts that actual QoS may deviate from the initial estimates, a replanning is triggered. To prevent risks at an earlier stage, the triggering algorithm permits an early activation of the replanning algorithm. Once the algorithm is invoked, the execution of workflow is stopped and replanning takes place on a slice of the workflow involving nodes which are not yet executed. The workflow resumes after the replanning.

4.3.2 QoS-Aware Middleware for Web Services Composition

Zeng *et al*. [21] present a middleware platform known as Agflow which addresses i) QoS modelling ii) QoS aware composition of web services and iii) composition service execution in a dynamic environment. QoS modelling takes into account the multi-dimensional properties in web services. Given a set of user requests and a set of candidate component services, selection of a component service is done in a way which optimises the composite service QoS. The third component takes care of run-time changes in QoS of the component services by replacing the services so that QoS is optimal.

4.4 Monitoring For Faults

Monitoring of composition is performed for runtime correctness analysis. When the composition is executed, the pre-defined properties are evaluated to check if they are satisfied. Events which lead to violations trigger the need for composition adaptation.

4.4.1 Assumption Based Verification for SBA Adaptation

Gehlert *et al.* [22] specify that individual service failure may or may not lead to violation of SBA requirements. However they also argue that monitoring the whole SBA helps to identify requirement deviations, but it may not be able to provide information about the failures which caused the deviation, which is an important factor for adaptation. The authors propose a formal verification technique for monitoring individual services which helps to identify whether a problem leads to a violation of requirement, and traces the violation to its root cause. Assumptions are based on requirements of the SBA; whenever assumptions are not met, it verifies whether the requirements can still be met or leads to a problem which may trigger adaptation pro-actively before the failure occurs.

4.4.2 Smart Monitors for Composed Services

Baresi *et al.* [23] propose three mechanisms for monitoring of composition specified by BPEL processes. The monitors are themselves web services. The three mechanisms correspond to the three behaviours which are (i) time outs, (ii) run-time errors and (iii) violation of functional contracts specified as assertions. The assertions specify the pre and post-conditions on service invocations. The control assertions are specified using process annotations that are translated into code extensions which interact with the monitor to perform assertion checking.

4.4.2 Run time monitoring instances and classes

Barbon *et al.* [24] have devised an architecture which supports monitoring single instances of BPEL processes known as Instance monitors and aggregated information on all instances known as Class monitors. The monitors are services implemented in BPEL, which run in parallel with the BPEL engine, interpret the input-output messages sent by the processes and detect misbehaviour. The authors have devised a monitoring specification language (RTML) which can be automatically translated into Java code.

4.5 Service Infrastructure Monitoring

In service infrastructure monitoring the monitoring components are the run-time environments required for the service execution.

4.5.1 A Layered Approach for SLA-Violation Propagation in Cloud Infrastructures

Brandic *et al.* [25] propose a layered approach for the management of cloud infrastructure to comply with user requirements specified in SLA. The main components of the architecture are meta-negotiator, meta-broker, broker, automatic service deployer and the resource. For detecting the source of failure, low level resource metrics are mapped to high level SLA parameters. The generation of SLA is done in a top down fashion whereas the management of SLA violation is done in a bottom up fashion. An SLA manager is present in each layer whose two main components are an autonomic manager and a notification broker. The autonomic manager consists of a knowledge database used for decision making by utilising case based reasoning. If the SLA manager identifies that the threat cannot be handled in the particular layer, it then publishes the problem to the layer above it. In the worst case the propagation may end up in the top most layer which informs the user about the problem and re-negotiates or aborts execution.

4.5.2 Performance Monitoing of SLAs for Utility Computing

Farrell *et al.* [26] define ontology to capture aspects of SLA in the domain of utility computing and deal with performance monitoring of contracts. The approach is based on event calculus. The contracts defined on the basis of contract patterns are axiomatised using event calculus. The effects of critical events on contract state are defined, which can be used for carrying out prediction.

5. Conclusion

In this review, we have presented a classification of monitoring techniques in Service-Based Applications. We have presented the monitoring aspects performed in the three SBA layers in isolation. We conclude that the current approaches to monitoring followed by adaptation are specific to each layer and are fragmented. They only address either specific problems, particular application domains or are limited to particular types of applications. We find that there is a gap in integration and alignment of monitoring events

across the layers of SBA. The monitoring mechanism across the functional layers should be coordinated.

REFERENCES

[1] R Kazhamiakin, M Pistore and A Zengin, "Cross-layer adaptation and monitoring of service-based applications," Proc. Int. Conf. Service-Oriented Computing (ICSOC 09), Springer, Nov. 2009, pp. 325-334, doi: 10.1007/978-3-642-16132-2_31.

[2] D. Peters, "Automated testing of real-time systems", Tech. Rep., Memorial Univ. of Newfoundland, 1999.

[3] B. Plattner and J. Nievergelt, "Monitoring program execution: A survey," Computer, vol. 14, Nov. 1981, pp. 76–93, doi: 10.1109/C-M.1981.220255.

[4] N. Delgado, A. Q. Gates, and S. Roach, "A taxonomy and catalog of runtime software-fault monitoring tools," IEEE Trans. Softw. Eng., vol. 30, Dec. 2004, pp. 859–872, doi: 10.1109/TSE.2004.91.

[5] C. Ghezzi and S. Guinea, "Run-time monitoring in service-oriented architectures," Test and Analysis of Web Services, Springer, 2007, pp. 237–264, doi: 10.1007/978-3-540-72912-9_9.

[6] J. Hielscher, A. Metzger and R. Kazhamiakin, "Taxonomy of adaptation principles and mechanisms", S-Cube project deliverable: CD-JRA-1.2.2. [Online]. Available: http://www.s-cube-network.eu/achievements-results/s-cube-deliverables.

[7] W. M. P. van der Aalst, A. H. M. ter Hofstede, and M. Weske. "Business process management: A survey," Proc. of the 1st Int. Conf. Business Process Management, vol. 2678, Springer, 2003, pp. 1–12, doi: 10.1007/3-540-44895-0_1.

[8] D. McCoy, R. Schulte, F. Buytendijk, N. Rayner and A. Tiedrich, "Business Activity Monitoring: The Promise and Reality", Gartner, Gartner's Marketing Knowledge and Technology COM-13-9992, Jul. 2001.

[9] D. Luckham, The power of events: An introduction to complex event processing in distributed enterprise systems, 1st ed. Addison-Wesley Professional, 2002.

[10] S. Srinivasan, V. Krishna and S. Holmes. "Web-log-driven Business Activity Monitoring," Computer, vol. 38, Mar. 2005, pp. 61-68, doi:10.1109/MC.2005.109.

[11] M. Castellanos, F. Casati, M. Shan and U. Dayal, "iBOM: A platform for intelligent business operation management," Proc. Int. Conf. Data Eng. (ICDE 05), Apr. 2005, pp. 1084-1095, doi:10.1109/ICDE.2005.73.

[12] K. H. Han, S. H. Choi, J. G. Kang and G. Lee, "Business Activity Monitoring system design framework integrated with process-based performance measurement model," WSEAS Transactions on Information Science And Applications, WEEAS, vol. 7, Mar. 2010, pp. 443-452,

[13] C. Pedrinaci and J. Domingue, "Ontology-based metrics computation for business process analysis," Proc. 4th Int. Workshop Semantic Business Process Management (SBPM 09), ACM Press, May 2009, pp. 43-50, doi: 10.1145/1944968.1944976.

[14] A. Ortega, M. Resinas, and A. Ruiz-Cort´es, "Defining process performance indicators: An ontological approach," Proc. Int. Conf. The Move to Meaningful Internet Systems - Volume Part I, (OTM 10), Springer, pp. 555–572, 2010, doi:10.1007/978-3-642-16934-2_41.

[15] J. Jeng, J. Schiefer and H. Chang, "An Agent-based architecture for analyzing business processes of real-time enterprises," Proc. 7th Int. Conf. Enterprise Distributed Object Computing (EDOC 03), IEEE Computer Society, Sept. 2003, pp. 86-97, doi: 10.1109/EDOC.2003.1233840.

[16] L. Ardissono, R. Furnari, A. Goy, G. Petrone and M. Segnan, "Fault tolerant web service orchestration by means of diagnosis," Proc. 3rd European Conf. Software Architecture (EWSA 06), Springer, 2006, pp. 2-16, doi:10.1007/11966104_2.

[17] A. Sahai, V. Machiraju, M. Sayal, A. Moorsel and F. Casati, "Automated SLA monitoring for web services", Proc. 13th IFIP/IEEE Int. Workshop Distributed Systems: Operations and Management: Management Technologies for E-Commerce and E-Business Applications (DSOM '02), Springer, Oct. 2002, pp. 28-41, doi: 10.1007/3-540-36110-3_6.

[18] H. Ludwig, A. Dan and R. Kearney, "Cremona: An architecture and library for creation and monitoring of WS-Agreements," Proc. Int. Conf. Service Oriented computing (ICSOC'04), ACM Press, 2004, pp. 65-74, doi.10.1145/1035167.1035178.

[19] K. Mahbub and G. Spanoudakis, "Monitoring WS-Agreements: An event calculus based approach," Test and Analysis of Web Services, Springer, 2007, pp 265-306, doi: 10.1007/978-3-540-72912-9_10.

[20] G. Canfora, M. Penta, R. Esposito and M. Villani, "QoS-Aware replanning of composite web services," Proc. IEEE Int. Conf. Web Services (ICWS 05), IEEE Computer Society, Jul. 2005, pp. 121-129, doi:10.1109/ICWS.2005.96.

[21] L. Zeng, B. Benatallah, A. Ngu, M. Dumas, J. Kalagnanam, and H. Chan, "QoS-Aware middleware for web services composition", IEEE Trans. Softw. Eng., vol. 30, IEEE Computer Society, May 2004, pp. 311-327, doi: 10.1109/TSE.2004.11.

[22] A. Gehlert, A. Bucchiarone, R. Kazhamiakin, A. Metzger, M. Pistore and K. Pohl, "Exploiting assumption based verification for the adaptation of SBA," Proc. ACM Symp. Applied Computing (SAC 10), ACM Press, Mar 2010, pp. 2430-2437, doi:10.1145/1774088.1774593.

[23] L. Baresi, C. Ghezzi and S. Guinea, "Smart monitors for composed services," Proc. 2nd Int. Conf. Service Oriented Computing (ICSOC 04), ACM Press, pp. 193-202. 2004, doi:10.1145/1035167.1035195.

[24] F. Barbon, P. Traverso, M. Pistore and M. Trainotti, "Run-time monitoring of instances and classes of web Service compositions," Proc. IEEE Int. Conf. Web Services (ICWS 06), IEEE Computer Society, 2006, pp. 63-71, doi:10.1109/ICWS.2006.113.

[25] I. Brandic, V. Emeakaroha, S. Acs, A. Kertesz and G. Kecskemeti, "LAYSI: A layered approach for SLA-violation propagation in self-manageable cloud infrastructures," Proc. IEEE Comput. Softw. and Appl. Conf. Workshops (COMPSACW 10), 2010, IEEE Computer Society, pp. 365 – 370, doi:10.1109/COMPSACW.2010.70

[26] Farrell, M. Sergot, M. Salle and C. Bartolini, "Performance monitoring of service-level agreements for utility computing using the event calculus," Proc. 1st IEEE Int. Workshop Electronic Contracting (WEC '04). IEEE Computer Society, 2004, pp. 17-24, doi:10.1109/WEC.2004.15.